

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Обработка изображений в ИС»

Тема: «Обучение детекторов объектов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: осуществлять обучение нейросетевого детектора для решения задачи обнаружения заданных объектов.

Ход работы

1. Базируясь на своем варианте, ознакомиться с выборкой для обучения детектора, выполнить необходимые преобразования данных для организации процесса обучения (если это нужно!);
2. Для заданной архитектуры нейросетевого детектора организовать процесс обучения для своей выборки. Оценить эффективность обучения на тестовой выборке (mAP);
3. Реализовать визуализацию работы детектора из пункта 1 (обнаружение знаков на отдельных фотографиях из сети Интернет);
4. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	Детектор	Датасет
9	YOLO12s	Транспортные средства: https://universe.roboflow.com/roboflow-100/vehicles-q0x2v/dataset/2

Был изучен датасет транспортных средств vehicles-q0x2v, загруженный с Roboflow через зеркало Hugging Face. Выборка содержит 12 классов транспорта: big bus, big truck, bus-l-, bus-s-, car, mid truck, small bus, small truck, truck-l-, truck-m-, truck-s-, truck-xl-. Всего 4058 изображений, разбитых на обучающую (2634), валидационную (966) и тестовую (458) выборки. Данные уже представлены в формате YOLO TXT, каждому изображению соответствует текстовый файл с аннотациями, где указаны класс объекта и нормализованные координаты bounding box. Преобразования данных не потребовались, поскольку Roboflow отдаёт датасет сразу в формате, совместимом с Ultralytics. Единственная правка: в файле data.yaml относительные пути были заменены на абсолютные, потому что по умолчанию Ultralytics искал изображения в папке datasets/, а датасет лежал в корне проекта. После правки структура датасета была проверена скриптом download_dataset.py, подтверждено наличие всех папок и корректное количество файлов в каждой выборке.

Код программы:

```
download_dataset.py
import os
from huggingface_hub import snapshot_download

DATASET_DIR = "./vehicles-q0x2v"

def download_dataset():
    print("=" * 60)
    print("Загрузка датасета vehicles-q0x2v...")
    print("Это может занять 10-20 минут (~220 МБ)")
    print("=" * 60)

    snapshot_download(
        repo_id="LibreYOLO/vehicles-q0x2v",
        repo_type="dataset",
```

```

        local_dir=DATASET_DIR,
        local_dir_use_symlinks=False,
    )

    print(f"\nДатасет загружен: {os.path.abspath(DATASET_DIR)}")

def verify_dataset(dataset_path):
    print("\n--- Проверка структуры ---")

    for item in ["data.yaml", "train", "valid", "test"]:
        full = os.path.join(dataset_path, item)
        status = "OK" if os.path.exists(full) else "НЕ НАЙДЕН"
        print(f"    {status}: {item}")

    for split in ["train", "valid", "test"]:
        img_dir = os.path.join(dataset_path, split, "images")
        if os.path.isdir(img_dir):
            count = len([
                f for f in os.listdir(img_dir)
                if f.lower().endswith(('.jpg', '.jpeg', '.png'))
            ])
            print(f"    {split}: {count} изображений")

if __name__ == "__main__":
    download_dataset()
    verify_dataset(DATASET_DIR)
    print("\nГотово. Теперь запустите train.py")

train.py
import os
import yaml
from ultralytics import YOLO

DATASET_YAML = "./vehicles-q0x2v/data.yaml"
MODEL_NAME = "yolol2s.pt"
PROJECT_DIR = "runs/detect"
RUN_NAME = "vehicles_yolol2s"

EPOCHS = 8
IMGSZ = 416
BATCH = 4
DEVICE = "cpu"
WORKERS = 2

def check_data_yaml(path):
    if not os.path.exists(path):
        raise FileNotFoundError(
            f"Файл {path} не найден.\n"
            f"Сначала запустите download_dataset.py"
        )

    with open(path, 'r', encoding='utf-8') as f:
        data = yaml.safe_load(f)

    print("=" * 60)
    print("Содержимое data.yaml:")
    for key in ["path", "train", "val", "test", "nc", "names"]:
        if key in data:
            print(f"    {key}: {data[key]}")
    print("=" * 60)

    return data

def train():
    check_data_yaml(DATASET_YAML)

```

```

print(f"\nЗагрузка модели: {MODEL_NAME}")
print("(при первом запуске веса скачаются автоматически, ~19 МБ)")
model = YOLO(MODEL_NAME)

print("\n" + "=" * 60)
print("НАЧАЛО ОБУЧЕНИЯ НА CPU")
print(f"    Эпох:          {EPOCHS}")
print(f"    Размер:         {IMGSZ}")
print(f"    Batch:          {BATCH}")
print(f"    Устройство:     {DEVICE}")
print("ВНИМАНИЕ: обучение на CPU может занять несколько часов!")
print("=" * 60 + "\n")

model.train(
    data=DATASET_YAML,
    epochs=EPOCHS,
    imgsz=IMGSZ,
    batch=BATCH,
    device=DEVICE,
    workers=WORKERS,
    optimizer='SGD',
    lr0=0.01,
    patience=10,
    project=PROJECT_DIR,
    name=RUN_NAME,
    exist_ok=True,
    pretrained=True,
    verbose=True,
    cache=False,
)

print(f"\nОбучение завершено: {PROJECT_DIR}/{RUN_NAME}")

def validate():
    best = os.path.join(PROJECT_DIR, RUN_NAME, "weights", "best.pt")

    if not os.path.exists(best):
        print(f"best.pt не найден: {best}")
        return

    print(f"\nВалидация на тестовой выборке: {best}")
    model = YOLO(best)

    metrics = model.val(
        data=DATASET_YAML,
        split='test',
        imgsz=IMGSZ,
        conf=0.25,
        iou=0.7,
        device=DEVICE,
    )

    print("\n" + "=" * 60)
    print("РЕЗУЛЬТАТЫ НА ТЕСТОВОЙ ВЫБОРКЕ")
    print("=" * 60)
    print(f"    mAP@50:         {metrics.box.map50:.4f}")
    print(f"    mAP@50-95:      {metrics.box.map:.4f}")
    print(f"    Precision:      {metrics.box.mp:.4f}")
    print(f"    Recall:         {metrics.box.mr:.4f}")
    print("=" * 60)

if __name__ == "__main__":
    train()
    validate()

```

detect.py

```

import os
import sys

```

```

import cv2
import numpy as np
import requests
from ultralytics import YOLO

WEIGHTS = "runs/detect/vehicles_yolo12s/weights/best.pt"
OUTPUT_DIR = "detection_results"
CONF_THRESHOLD = 0.2
DEVICE = "cpu"

def load_model(weights_path):
    if not os.path.exists(weights_path):
        raise FileNotFoundError(
            f"Веса не найдены: {weights_path}\n"
            f"Сначала запустите train.py и дождитесь завершения обучения."
        )
    print(f"Загрузка модели: {weights_path}")
    return YOLO(weights_path)

def download_image(url):
    print(f"Скачивание изображения: {url}")
    headers = {
        "User-Agent": (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
            "AppleWebKit/537.36 (KHTML, like Gecko) "
            "Chrome/120.0.0.0 Safari/537.36"
        )
    }
    response = requests.get(url, headers=headers, timeout=30)
    response.raise_for_status()

    img_array = np.frombuffer(response.content, np.uint8)
    img = cv2.imdecode(img_array, cv2.IMREAD_COLOR)

    if img is None:
        raise ValueError("Не удалось декодировать изображение")

    return img

def detect_and_save(model, image, source_name):
    print("Выполняется детекция...")
    results = model(image, conf=CONF_THRESHOLD, device=DEVICE, verbose=False)
    result = results[0]

    annotated = result.plot()

    os.makedirs(OUTPUT_DIR, exist_ok=True)
    out_path = os.path.join(OUTPUT_DIR, f"{source_name}_result.jpg")
    cv2.imwrite(out_path, annotated)

    boxes = result.boxes
    if boxes is not None and len(boxes) > 0:
        print(f"\nОбнаружено объектов: {len(boxes)}")
        names = result.names
        for box in boxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            print(f" [{names[cls_id]}] "
                  f"уверенность={conf:.3f} "
                  f"bbox=({x1:.0f},{y1:.0f},{x2:.0f},{y2:.0f})")
    else:
        print("\nОбъекты не обнаружены. Попробуйте уменьшить CONF_THRESHOLD.")

    print(f"\nРезультат сохранён: {out_path}")
    return result

```

```

def detect_from_url(model, url):
    img = download_image(url)
    name = url.split("/")[-1].split("?")[0]
    name = os.path.splitext(name)[0][:30] or "web_image"
    return detect_and_save(model, img, name)

def detect_from_file(model, path):
    img = cv2.imread(path)
    if img is None:
        raise ValueError(f"Не удалось прочитать файл: {path}")
    name = os.path.splitext(os.path.basename(path))[0]
    return detect_and_save(model, img, name)

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Использование:")
        print("  python detect.py <URL изображения>")
        print("  python detect.py --file <путь к файлу>")
        print()
        print("Пример для PyCharm:")
        print("  Run → Edit Configurations → Parameters:")
        print("  https://example.com/traffic.jpg")
        sys.exit(1)

    model = load_model(WEIGHTS)

    if sys.argv[1] == "--file":
        if len(sys.argv) < 3:
            print("Укажите путь к файлу после --file")
            sys.exit(1)
        detect_from_file(model, sys.argv[2])
    else:
        detect_from_url(model, sys.argv[1])

```

Результат программы:

Программа `download_dataset.py` вывела в консоль информацию о загрузке датасета транспортных средств с Hugging Face. Появилось сообщение о начале загрузки с предупреждением, что это займёт 10-20 минут и объём датасета около 220 МБ. Затем отобразился прогресс-бар, показавший скачивание 8121 файла и достижение 100%. Также появилось уведомление об устаревшем параметре, которое не является ошибкой. После загрузки программа сообщила, что датасет успешно сохранён, и указала путь к нему. Далее запустилась проверка структуры, которая подтвердила наличие файла `data.yaml`, папок `train`, `valid`, `test` и корректное количество изображений в каждой: 2634, 966 и 458 соответственно. В конце программа вывела сообщение, что всё готово и можно запускать `train.py`. Процесс завершился без ошибок. Единственная проблема по ходу – ошибка 429 от Hugging Face после скачивания 8113 файлов из 8121, вызванная временной блокировкой IP за частые запросы. Она решилась регистрацией на Hugging Face, созданием токена и повторным запуском скрипта, который докачал оставшиеся файлы.

Программа `train.py` обучила модель YOLO12s на датасете транспортных средств за 8 эпох. В процессе обучения выводились метрики по каждой эпохе: значения функции потерь (`box_loss`, `cls_loss`, `dfl_loss`) снижались, а точность (mAP) росла. Обучение заняло 3 часа 38 минут на CPU. После завершения модель была проверена на валидационной и тестовой

выборках. Лучший результат показала 8-я эпоха, её веса сохранены как best.pt. На тестовой выборке итоговые метрики: mAP@50 = 0,3837, mAP@50-95 = 0,2718, Precision = 0,4747, Recall = 0,4038. Лучше всего модель определяет класс car (mAP@50 = 0,76), big bus (0,67), big truck (0,54) и bus-s- (0,53). Слабее — редкие классы small bus, truck-s-, mid truck, что объясняется малым числом примеров в датасете. Процесс завершился без ошибок.

Динамика обучения по эпохам

Эпоха	mAP@50	mAP@50-95
1	0,233	0,137
2	0,261	0,164
3	0,336	0,209
4	0,365	0,233
5	0,357	0,233
6	0,419	0,280
7	0,412	0,278
8	0,411	0,278

Для визуализации использован скрипт detect.py, который принимает URL изображения или путь к локальному файлу, прогоняет через обученную модель YOLO12s и сохраняет результат с bounding box. Были проверены три изображения. На них детектор обнаружил 13, 5 и 7 объектов соответственно. Классы: car, truck-l-, truck-m-, truck-s-, truck-xl-, big truck. Максимальная уверенность — 0,916 для класса truck-xl-. Результаты сохранены в папке detection_results/.

```
python detect.py --file C:\Users\admin\PycharmProjects\OIS_LAB3\vehicles-q0x2v\test\images\adit_mp4-328_jpg.rf.bc62a502f8b2f5fdb62fafef49f2d274.jpg
```

```
[truck-l-] уверенность=0.719 bbox=(396,294,490,435)
```

```
[truck-l-] уверенность=0.710 bbox=(410,238,473,311)
```

```
[car] уверенность=0.695 bbox=(323,343,369,417)
```

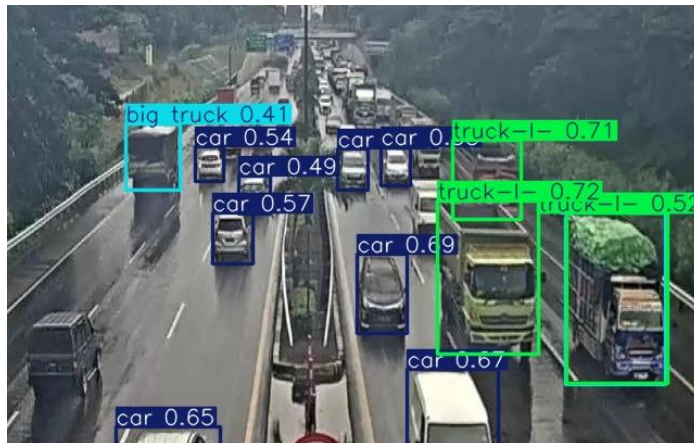
```
[car] уверенность=0.666 bbox=(369,450,453,597)
```

```
[car] уверенность=0.648 bbox=(101,502,196,623)
```

```
[car] уверенность=0.569 bbox=(189,305,226,353)
```

[car] уверенность=0.543 bbox=(174,244,199,276)
 [car] уверенность=0.529 bbox=(344,245,372,281)
 [truck-l-] уверенность=0.524 bbox=(516,306,608,463)
 [car] уверенность=0.494 bbox=(214,272,243,306)
 [big truck] уверенность=0.429 bbox=(515,306,606,464)
 [big truck] уверенность=0.412 bbox=(109,223,160,284)
 [car] уверенность=0.404 bbox=(304,246,333,284)

detection_results\adit_mp4-328_jpg.rf.bc62a502f8b2f5fdb62fafef49f2d274_result.jpg



python detect.py --file C:\Users\admin\PycharmProjects\OIIS_LAB3\images\cars1.jpg

[truck-xl-] уверенность=0.916 bbox=(184,254,228,321)
 [car] уверенность=0.688 bbox=(103,564,199,640)
 [car] уверенность=0.662 bbox=(162,324,200,374)
 [truck-m-] уверенность=0.583 bbox=(16,396,129,532)
 [car] уверенность=0.455 bbox=(251,266,269,288)

detection_results\cars1_result.jpg



python detect.py --file C:\Users\admin\PycharmProjects\OIIS_LAB3\images\cars2.jpg

[car] уверенность=0.791 bbox=(318,287,341,318)

[car] уверенность=0.765 bbox=(242,284,263,313)
[car] уверенность=0.760 bbox=(401,292,430,328)
[truck-m-] уверенность=0.528 bbox=(197,269,225,308)
[truck-m-] уверенность=0.439 bbox=(346,270,377,314)
[truck-s-] уверенность=0.421 bbox=(346,270,377,314)
[truck-s-] уверенность=0.227 bbox=(197,268,225,307)

detection_results\cars2_result.jpg



Вывод: осуществила обучение нейросетевого детектора для решения задачи обнаружения заданных объектов.